

使用陣列

•	20-1:常用的陣列屬性	
	常用屬性.....	20-1
•	20-2:System.Array類別常用的方法	
	陣列類別常用方法.....	20-2
•	20-3:傳回陣列的方法 / 傳遞陣列當做參數	
	有傳回值的方法.....	20-4
	傳回陣列的方法.....	20-5
	傳遞陣列當做參數.....	20-6
	使用foreach迴圈處理陣列元素.....	20-7
	不規則陣列.....	20-9

Module 20

常用屬性

- 以下為陣列所提供的常用屬性：

- **ArrayObject.Rank**

- 取得陣列的維度數量。

```
int[,] arr = new int[2,3];  
Console.WriteLine(arr.Rank); // 2
```

- **ArrayObject.Length**

- 取得陣列的長度。

```
int[] arr = new int[5];  
Console.WriteLine(arr.Length); // 5
```

陣列類別常用方法-1

- **ArrayObject.GetLength(int dimension) :**
 - 取得指定陣列維度的長度(參數0為第一維度，以此類推)。

```
int[,] arr = new int[2, 3];  
Console.WriteLine(arr.GetLength(0)); //2
```

- **Array.IndexOf(Array arr, object val) :**

- 取得指定元素第一個在陣列中的索引位置。

```
int[] arr = { 4, 8, 2, 10, 6 };  
Console.WriteLine(Array.IndexOf(arr, 10)); //3
```

- **Array.LastIndexOf(Array arr, object val) :**

- 取得指定元素最後一個在陣列中的索引位置。

```
int[] arr = { 4, 8, 2, 10, 6, 10 };  
Console.WriteLine(Array.LastIndexOf(arr, 10)); //5
```

陣列類別常用方法-2

- **Array.Sort(Array arr) :**

- 排序陣列的方法。

```
int[] arr = { 4, 8, 2, 10, 6 };  
Array.Sort(arr);
```

- **Array.Clear(Array arr, int index, int length) :**

- 變更陣列指定範圍的元素為初始值(index為開始位置，length為清除長度)。

```
int[] arr = { 4, 8, 2, 10, 6, 10 };  
Array.Clear(arr, 0, 6);
```

- **Array.Copy(Array arr, Array newarr, int length) :**

- 複製陣列到指定陣列的方法。

```
int[] arr = { 4, 8, 2, 10, 6, 10 };  
int[] newarr = new int[5];  
Array.Copy(arr, newarr, 3);
```

有傳回值的方法

- 沒有傳回值的方法：

```
internal void MyVoid()
{
    Console.WriteLine("我有回傳值啦");
}
```

- 有傳回值的方法：

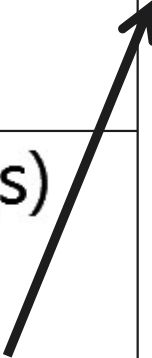
```
static void Main(string[] args)
{
    int result = MyInt(10);
    Console.WriteLine(result);
    Console.ReadKey();
}
```



```
private static int MyInt(int i)
{
    int a = i + 10;
    return a; //必加return
}
```

傳回陣列的方法

```
static void Main(string[] args)
{
    int[] arr = CreateArray(69);
    Console.WriteLine(arr.Length); // 69
    Console.ReadKey();
}
```

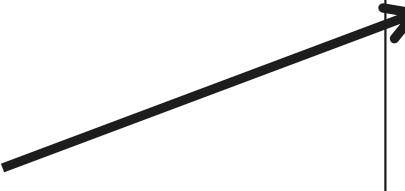


```
static int[] CreateArray (int size)
{
    int[] create = new int[size];
    return create;
}
```

傳遞陣列當做參數

- 陣列為參考型別，當帶入參數時是傳入記憶體位置而非值，所以在方法中修改參數將會影響原變數值。

```
static void Main(string[] args)
{
    int[] arr = {3,6,9 };
    ArrayMethod(arr);
    for(int i = 0; i<arr.Length; i++)
    {
        Console.WriteLine(arr[i]); // ?
    }
    Console.ReadKey();
}
```



```
static void ArrayMethod(int[] array)
{
    array[0] += 100;
}
```

使用foreach迴圈處理陣列元素-1

- foreach適合用來取用陣列或集合中的所有元素。
- 語法：
 - `foreach(元素型態 元素變數 in 陣列或集合) { //... }`
 - `foreach(int item in arr)`
`{`
`//...`
`}`

使用foreach迴圈處理陣列元素-2

- 比較for迴圈和foreach迴圈的差異。

```
static void Main(string[] args)
{
    int[] arr = {3,6,9 };
    ArrayMethod(arr);
    for(int i = 0; i<arr.Length; i++)
    {
        Console.WriteLine(arr[i]); // ?
    }
    Console.ReadKey();
}
```

```
static void Main(string[] args)
{
    int[] arr = { 3, 6, 9 };
    ArrayMethod(arr);
    foreach (int item in arr)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
```

不規則陣列

- 規則陣列或稱矩陣陣列(二維陣列)，相同維度的元素數量必須相同。
 - `int[,] arr = new int[2, 3];`
- 不規則陣列是C#提供的特殊二維陣列，陣列中的元素陣列長度可以不相同。

```
int[][] arr = new int[3][];  
arr[0] = new int[] { 1, 2, 3 };  
arr[1] = new int[] { 1, 2, 3, 4, 5 };  
arr[2] = new int[] { 1, 2 };
```